ESCUELA DE INGENIERÍA EN COMPUTACIÓN
INGENIERÍA EN DESARROLLO DE SOFTWARE

ING. GILBERT GARCÍA

EVALUACIÓN PRÁCTICA N.º 2 – IDS221

Gestión de Datos para Inteligencia Artificial y Big Data	
Integrantes	
1.	David Edgardo Lopez Salazar – 039125
2.	Jose Roberto Henriquez Calderon -- 098925
3.	Jean Carlo Arbizu Orellana -- 053925
4.	Fernando Dagoberto García Hernández --038325
5.	

INTEGRIDAD ACADÉMICA: Pueden consultar los materiales trabajados en clase y documentación técnica. Durante el examen no se permite delegar el desarrollo, el razonamiento ni las justificaciones a asistentes de IA generativa, terceros o soluciones ajenas. Las evidencias deben poder explicarse y reproducirse en una comprobación breve ante el docente. (Vídeo de 5 minutos máximo)

1. Competencia general

Implementar una solución analítica reproducible a partir de datos empresariales mediante visualización interactiva, análisis geoespacial, reportería automatizada y predicción básica para sustentar decisiones operativas.

2. Competencias a evaluar

Componente	Desempeño verificable	Guía
Dashboard de negocio	Construye medidas DAX, KPIs y segmentadores; comprueba resultados.	5
Análisis geoespacial	Valida coordenadas y configura capas y dos vistas territoriales.	6
Reportería automatizada	Regenera informe con KPIs, gráficos, plantilla y metadata.	7
Modelado predictivo	Compara baseline y regresión lineal con partición temporal y métricas.	8
Argumentación técnica	Defiende decisiones con evidencias reales de su ejecución.	Transversal

3. Caso de estudio: GreenEnergy Analytics

GreenEnergy Analytics continúa modernizando su operación energética en El Salvador. Después de integrar, limpiar y proteger datos (Evaluación Práctica 1), la gerencia requiere un tablero de cobertura energética, una lectura territorial de sus sedes, un informe que pueda actualizarse y un pronóstico básico de demanda diaria. El equipo consultor dispone de 180 días de operación sintética de nueve sedes, con una ubicación registrada incorrectamente. La empresa necesita decisiones sustentadas; no le sirve una presentación estética sin cifras verificables.

ROL: Ustedes actúan como equipo de Ingeniería en Desarrollo de Software contratado para entregar, en 24 horas, un prototipo analítico funcional y técnicamente defendible. Se trabajará con un solo conjunto de datos y los mismos requerimientos para todos los grupos.

4. Fuentes entregadas por la empresa

Archivo	Campos esenciales	Uso
operacion_sedes.csv	fecha, sede_id, region, generacion_kwh, demanda_kwh, evento_programado	Power BI, acumulación territorial y reporte
sedes_geograficas.csv	sede_id, nombre_sede, municipio, region, latitud, longitud	Mapa ArcGIS; incluye un registro para depurar
demanda_diaria.csv	fecha, demanda_kwh, evento_programado	Baseline, regresión lineal y evaluación temporal

5. Ruta sugerida de las 3 horas (sugerido)

Momento	Minutos	Producto
Lectura, roles y archivos	0–10	Equipo y fuentes verificados
Parte 1: Power BI	10–45	Dashboard + indicadores.csv
Parte 2: ArcGIS	45–75	CSV geográfico + 2 vistas
Parte 3: Reportería	75–115	Informe automatizado + metadata
Parte 4: Predicción	115–165	Predicciones + métricas
Integración y verificación	165–180	ZIP final, justificaciones, comprobación rápida

Se permite paralelizar tareas por integrante; sin embargo, todos deben comprender al menos una decisión y un resultado cuantitativo del producto global. Se debe consolidar en un solo entregable de grupo (Zip, repositorio y video)

6. Desarrollo de la evaluación

Se debe colocar captura de pantalla que evidencie cada paso

Parte 1. Dashboard interactivo con Power BI — 25 puntos

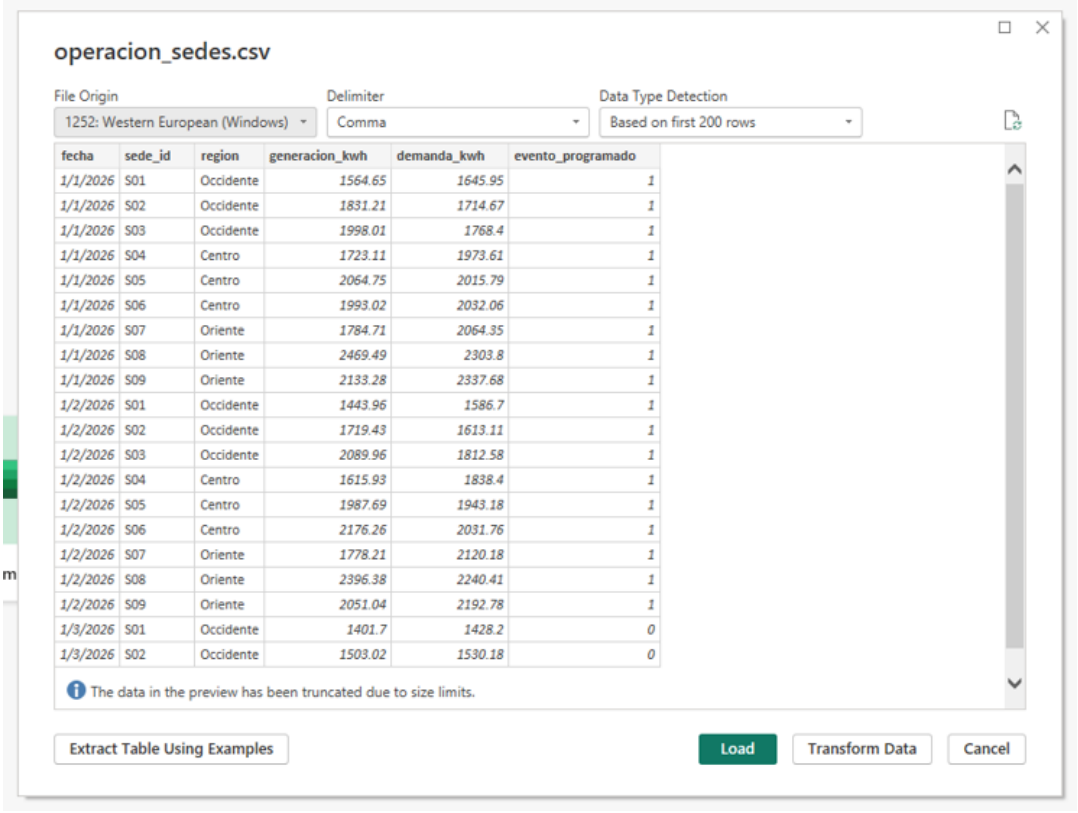
Problema: La gerencia desconoce qué proporción de la demanda queda cubierta con generación propia y en qué sedes se concentra el déficit.

- Importar operacion_sedes.csv en Power BI Desktop, comprobar tipos de fecha y datos numéricos.
- Construir cuatro medidas DAX:
  1. Energía generada total.
  2. Demanda total.
  3. Déficit energético total (suma de MAX(demanda − generación, 0) por registro).
  4. Cobertura (%) = 100 × (1 − déficit total / demanda total).
- Diseñar una página con las cuatro tarjetas KPI, un gráfico de evolución temporal y gráficos comparativos por región y sede; incorporar segmentador por región.
- Verificar una interacción: seleccionar una región y comprobar que cambian los indicadores; **guardar dashboard.pbix y vista_bi.png.**
- Entregar indicadores.csv con columnas indicador,valor; usar exactamente las cuatro claves del contrato de salida. Valores numéricos sin unidades y con punto decimal.

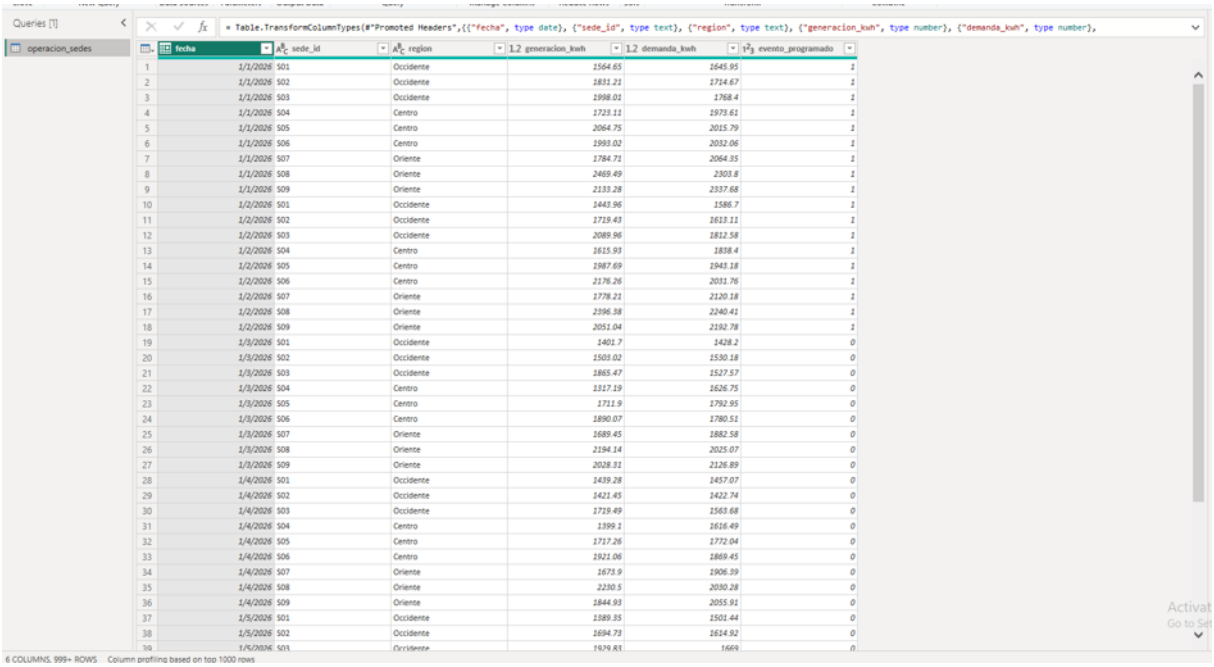
Justificación técnica (máximo 5 líneas): cite el déficit total y la cobertura % antes de filtrar. Explique qué medida no puede obtenerse sumando déficits netos globales y por qué el segmentador altera el contexto de cálculo.

Desarrollo parte 1 power BI:

Paso 1: Importamos operacion_sedes.csv y verificamos los datos



fecha	sede_id	region	generacion_kwh	demanda_kwh	evento_programado
1/1/2026	S01	Occidente	1564.65	1645.95	1
1/1/2026	S02	Occidente	1831.21	1714.67	1
1/1/2026	S03	Occidente	1998.01	1768.4	1
1/1/2026	S04	Centro	1723.11	1973.61	1
1/1/2026	S05	Centro	2064.75	2015.79	1
1/1/2026	S06	Centro	1993.02	2032.06	1
1/1/2026	S07	Oriente	1784.71	2064.35	1
1/1/2026	S08	Oriente	2469.49	2303.8	1
1/1/2026	S09	Oriente	2133.28	2337.68	1
1/2/2026	S01	Occidente	1443.96	1586.7	1
1/2/2026	S02	Occidente	1719.43	1613.11	1
1/2/2026	S03	Occidente	2089.96	1812.58	1
1/2/2026	S04	Centro	1615.93	1838.4	1
1/2/2026	S05	Centro	1987.69	1943.18	1
1/2/2026	S06	Centro	2176.26	2031.76	1
1/2/2026	S07	Oriente	1778.21	2120.18	1
1/2/2026	S08	Oriente	2396.38	2240.41	1
1/2/2026	S09	Oriente	2051.04	2192.78	1
1/3/2026	S01	Occidente	1401.7	1428.2	0
1/3/2026	S02	Occidente	1503.02	1530.18	0



En Power Query se revisó cada columna: fecha es tipo Fecha, generacion_kwh y demanda_kwh son Número decimal, evento_programado es Número entero y sede_id y region son Texto.

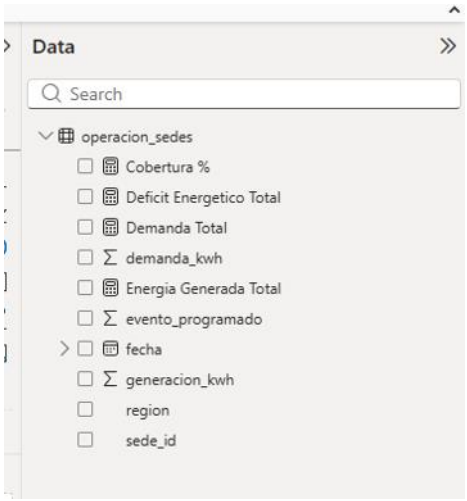
Paso 2: Medidas DAX

```
Energia Generada Total = SUM(operacion_sedes[generacion_kwh])

Demanda Total = SUM(operacion_sedes[demanda_kwh])

Deficit Energetico Total = SUMX(
  operacion_sedes,
  MAX(operacion_sedes[demanda_kwh] - operacion_sedes[generacion_kwh], 0)
)

Cobertura % = 100 * (1 - [Deficit Energetico Total] / [Demanda Total])
```



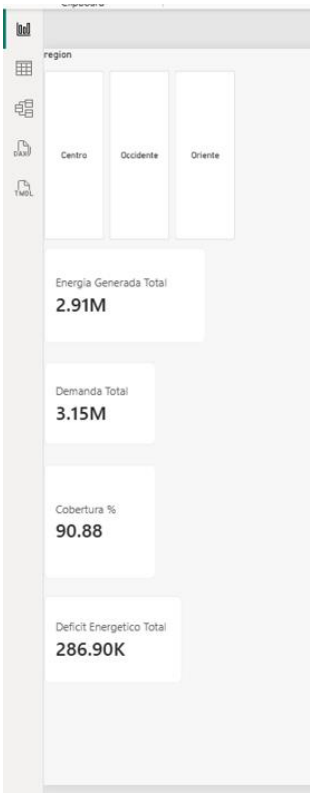
Paso 3. Resultados sin filtrar

Energía generada total: 2,913,623.55 kWh

Demanda total: 3,146,866.67 kWh

Déficit energético total: 286,904.20 kWh

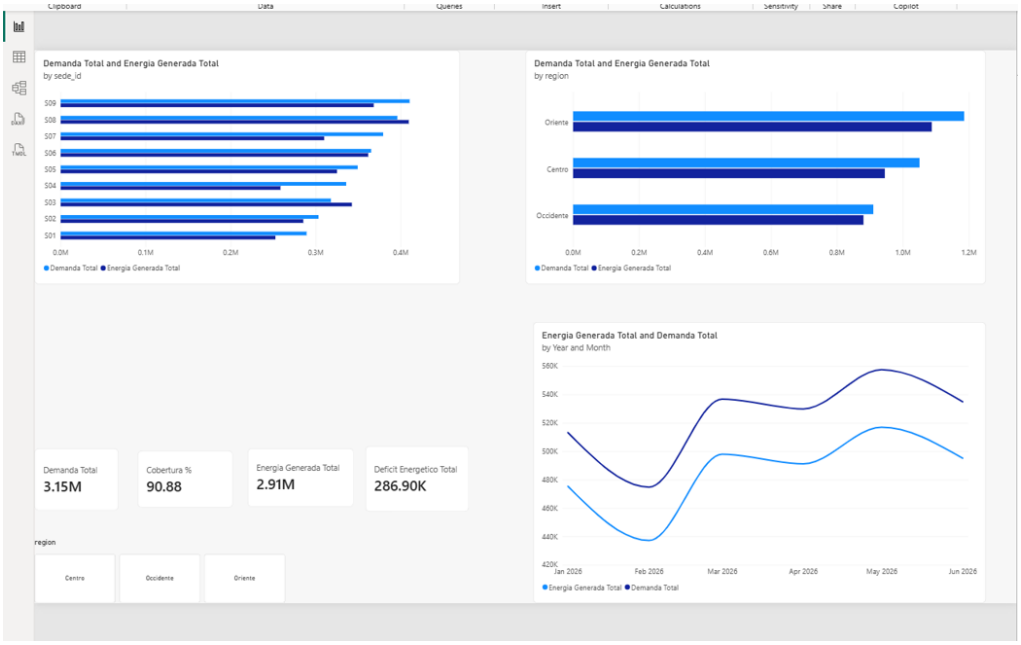
Cobertura: 90.88 %



Paso 4: Diseño visual

Dashboard completo:

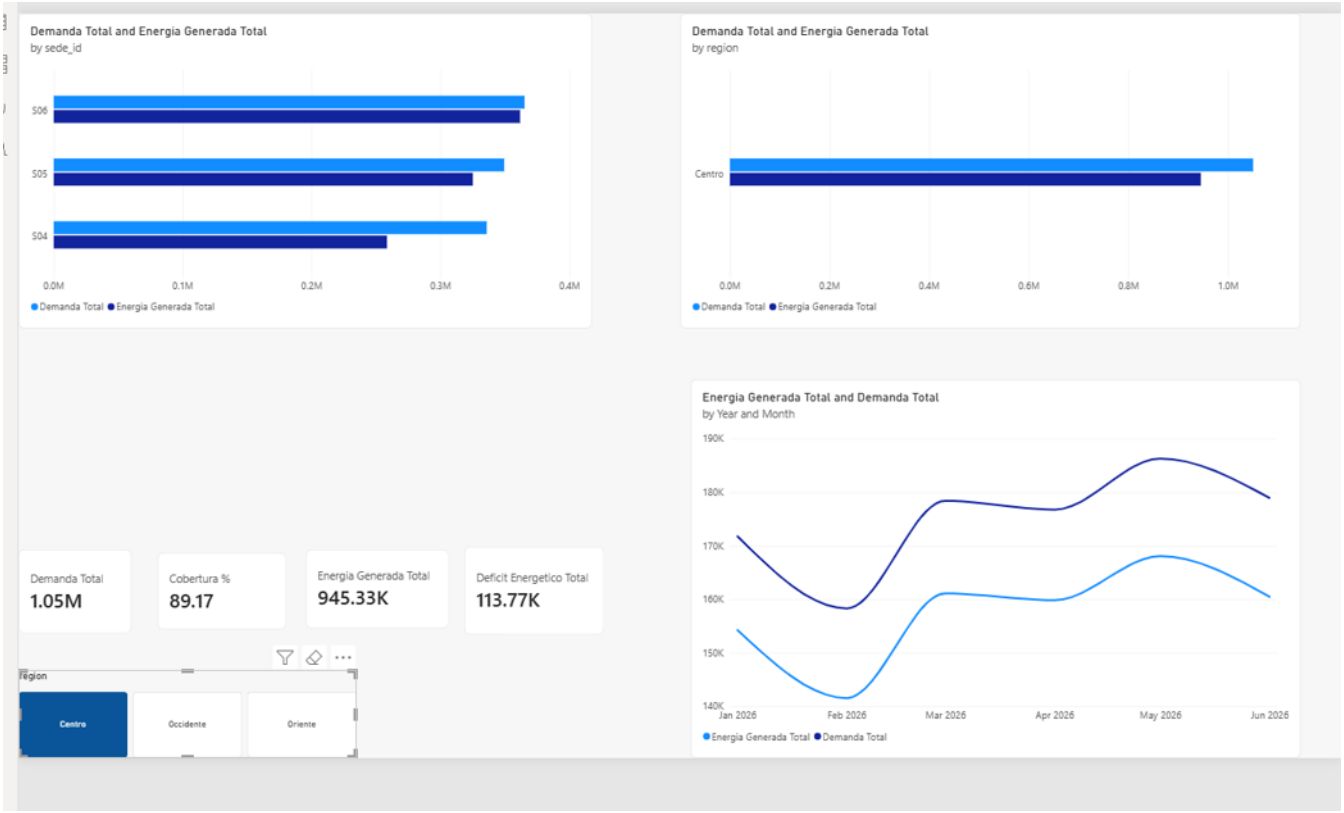
La página contiene cuatro tarjetas KPI, un gráfico de líneas de evolución temporal, gráficos de barras por región y por sede, y un segmentador por región.



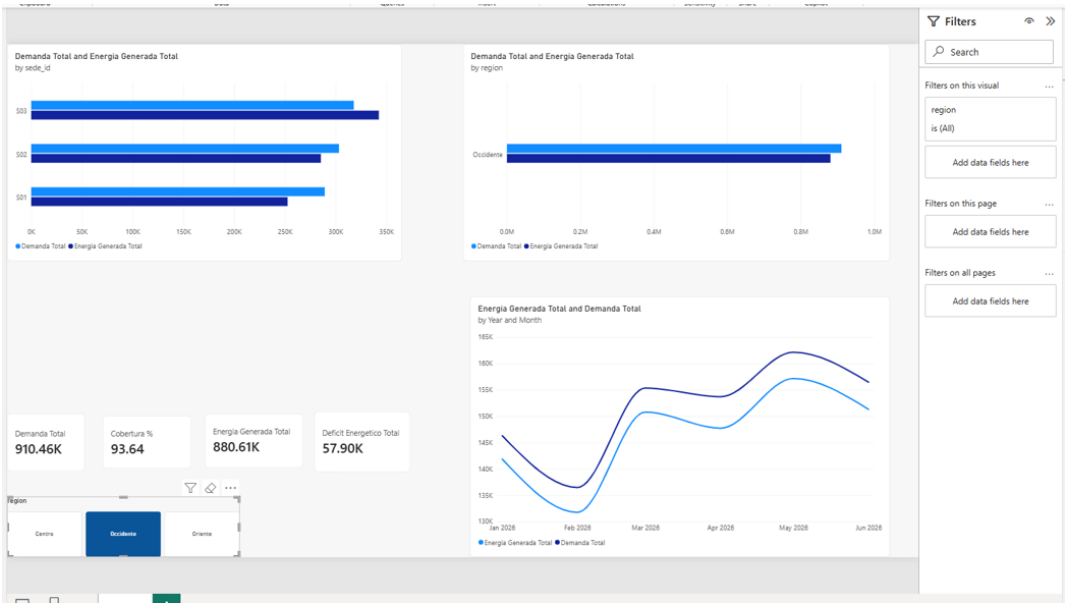
Paso 5: verificación interacción:

Al seleccionar la región Oriente en el segmentador, las tarjetas se recalculan: déficit 115,224.77 kWh y cobertura 90.28 %. Con Centro: 113,774.53 kWh y 89.17 %. Con Occidente: 57,904.90 kWh y 93.64 %.

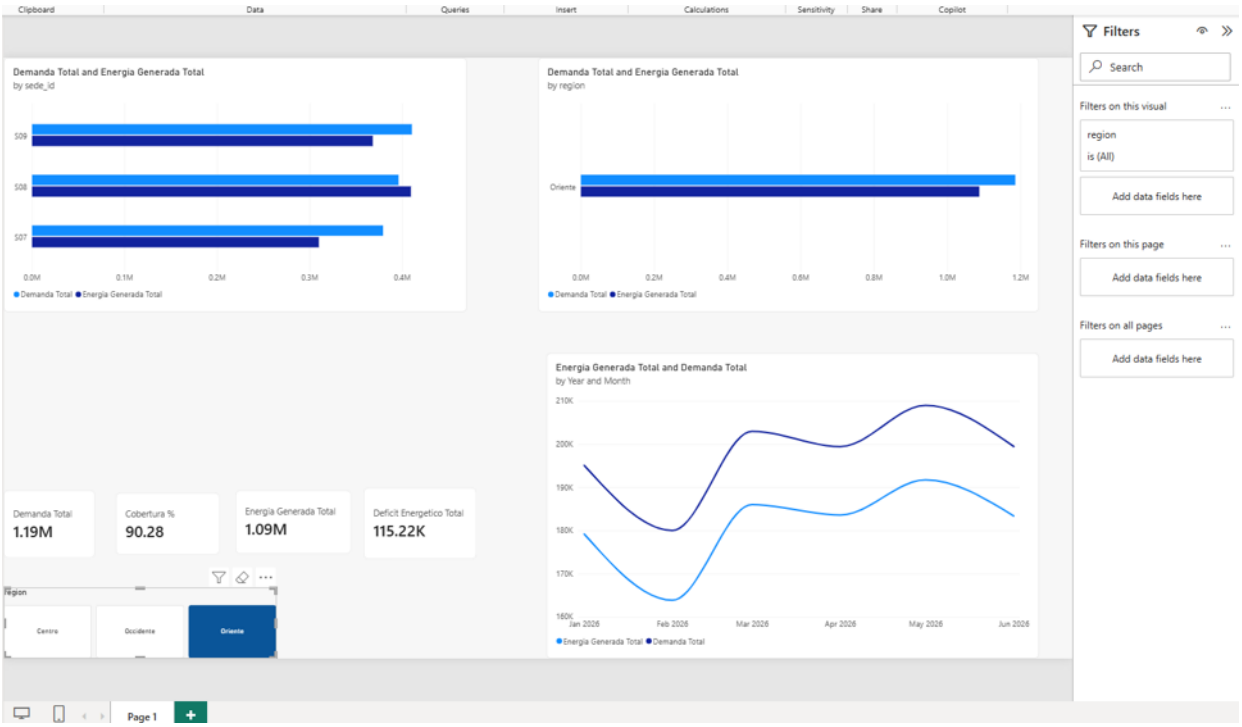
Con centro:



Con Occidente:

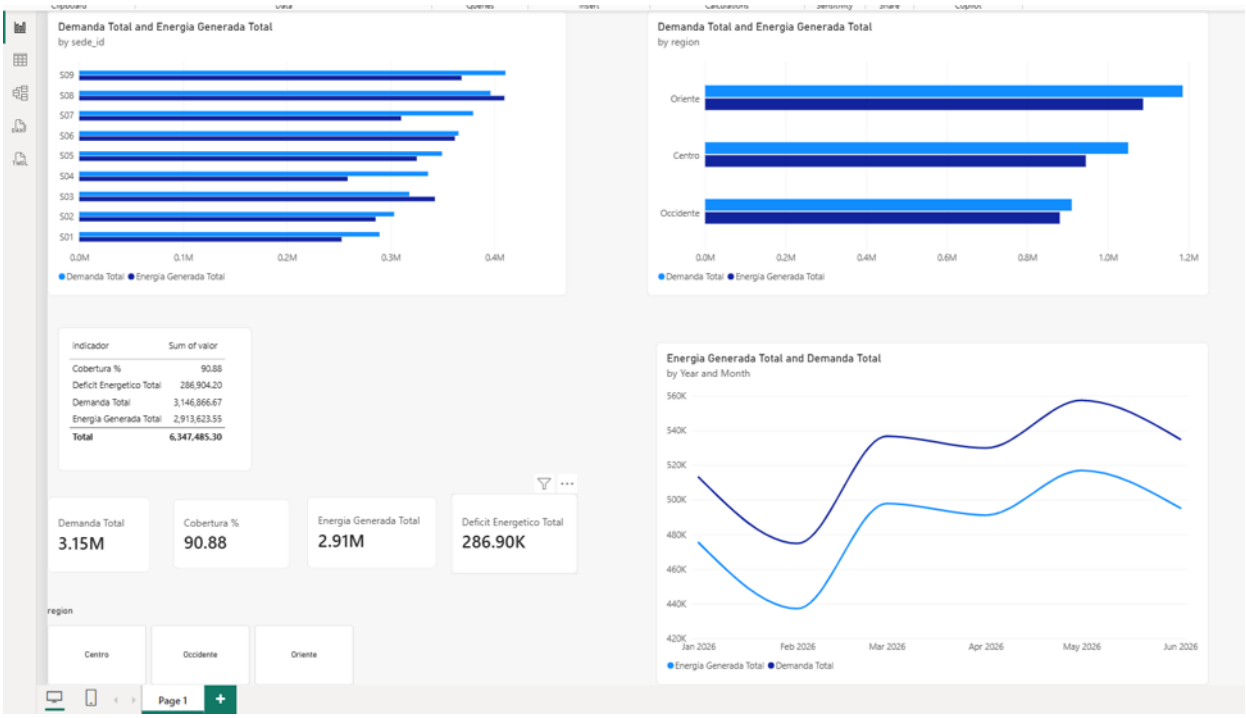


Con Oriente:



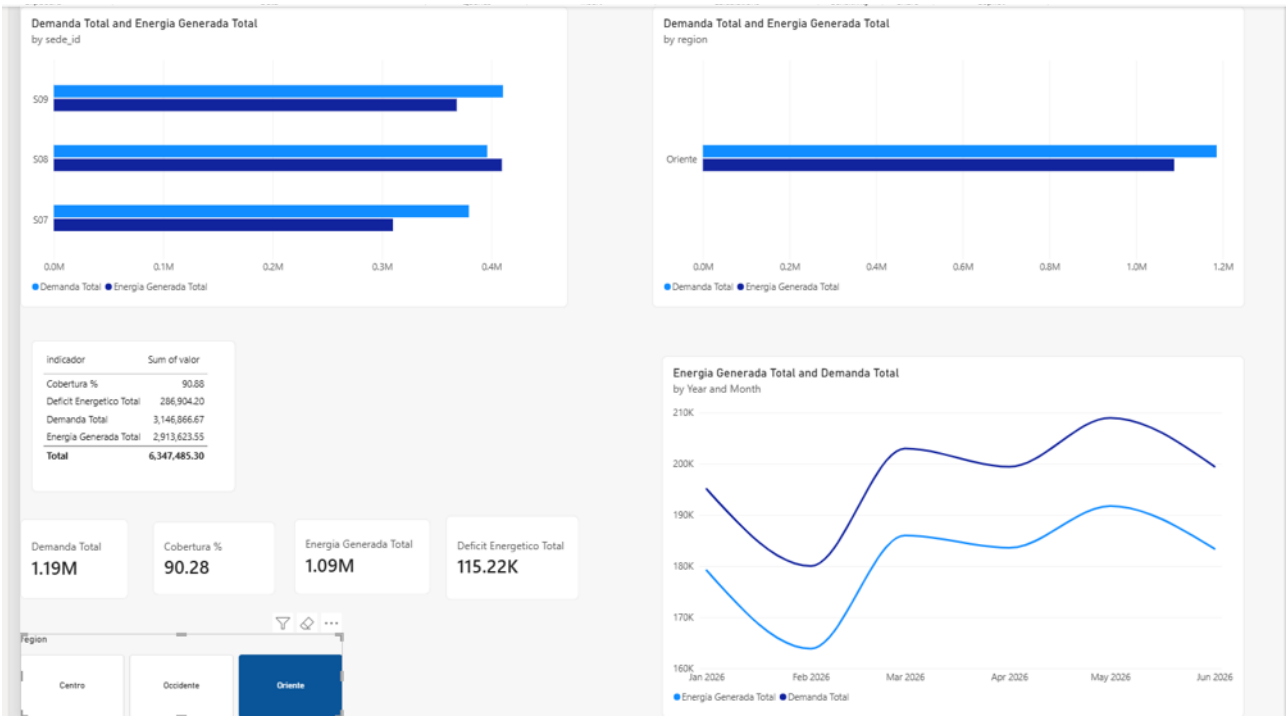
Paso 6: exportacion con indicadores.csv:

Se exportó indicadores.csv con columnas indicador,valor y las claves exactas energia_total_kwh, demanda_total_kwh, deficit_total_kwh y cobertura_pct, con punto decimal y sin unidades.



Justificacion técnica:

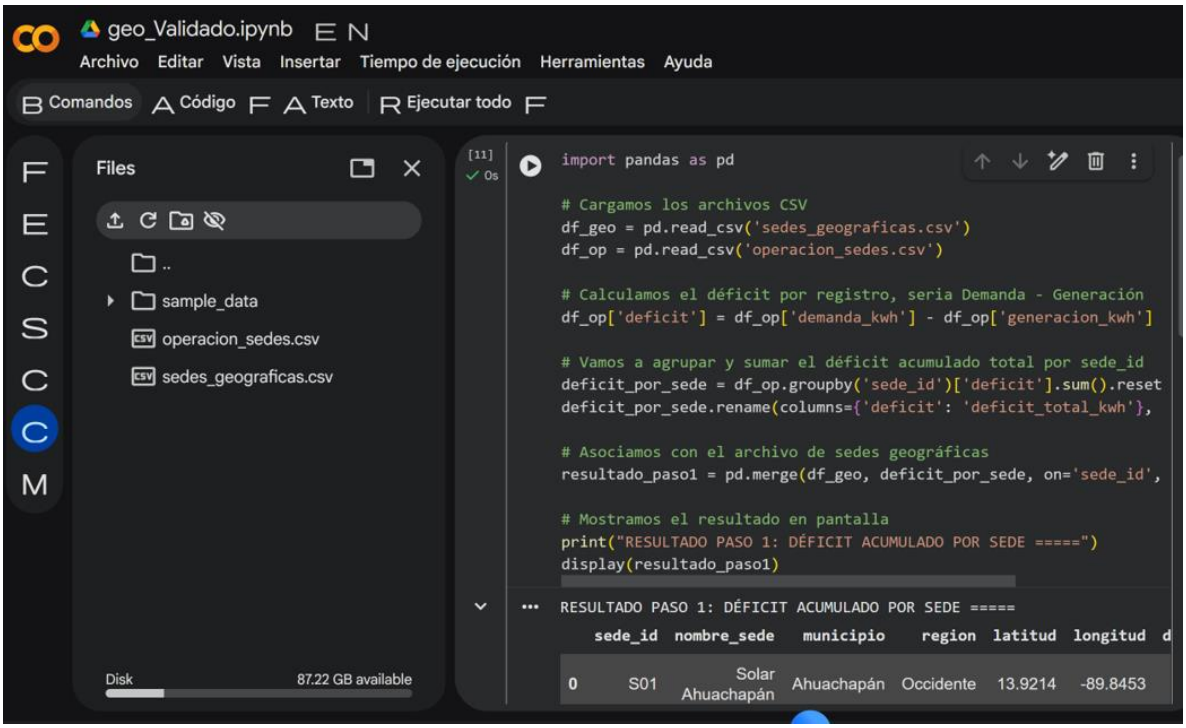
Antes de filtrar, el déficit total es 286,904.20 kWh y la cobertura es 90.88 %. El déficit no se puede sacar restando la demanda total menos la generación total (da 233,243.12 kWh y una cobertura falsa de 92.59 %), porque esa resta cubre los días que faltó energía con los días que sobró. Por eso se calcula fila por fila, tomando solo lo que faltó. Al elegir una región en el segmentador, Power BI calcula todo de nuevo solo con los datos de esa región: con Oriente, el déficit es 115,224.77 kWh y la cobertura 90.28 %.



Parte 2. Análisis geoespacial con ArcGIS — 20 puntos

Problema: La gerencia debe localizar sedes con déficit energético y detectar errores de ubicación antes de comunicar un mapa.

- Calcular con Python/Pandas el déficit acumulado por sede desde operacion_sedes.csv y asociarlo por sede_id con sedes_geograficas.csv.



RESULTADO PASO 1: DÉFICIT ACUMULADO POR SEDE

	sede_id	nombre_sede	municipio	region	latitud	longitud	d
0	S01	Solar Ahuachapán	Ahuachapán	Occidente	13.9214	-89.8453	
1	S02	Solar Santa Ana	Santa Ana	Occidente	13.9942	-89.5597	
2	S03	Solar Sonsonate	Sonsonate	Occidente	13.7189	-89.7241	
3	S04	Solar San Salvador	San Salvador	Centro	13.6929	-89.2182	
4	S05	Solar Santa Tecla	Santa Tecla	Centro	13.6769	-89.2797	
5	S06	Solar Cojutepeque	Cojutepeque	Centro	13.7179	-88.9331	
6	S07	Solar San Miguel	San Miguel	Oriente	13.4837	-88.1779	
7	S08	Solar Usulután	Usulután	Oriente	13.3500	-88.4461	
8	S09	Solar La	La Unión	Oriente	13.3369	-67.8440	

- Convertir coordenadas a numéricas y verificar plausibilidad territorial: 13 a 15 para latitud y −91 a −87 para longitud. Excluir del mapa los registros inválidos sin inventar coordenadas.

```
#PASO 2:
#•  Convertir coordenadas a numéricas y verificar plausibilidad
#territorial: 13 a 15 para latitud y -91 a -87 para longitud.
#Excluir del mapa los registros inválidos sin inventar coordenadas

# 2. CalculaMOS el déficit acumulado por sede
df_op['deficit'] = df_op['demanda_kwh'] - df_op['generacion_kwh']
deficit_por_sede = df_op.groupby('sede_id')['deficit'].sum().reset
deficit_por_sede.rename(columns={'deficit': 'deficit_total_kwh'},
resultado_paso1 = pd.merge(df_geo, deficit_por_sede, on='sede_id',

# 3. Convertimos coordenadas a numéricas
resultado_paso1['latitud'] = pd.to_numeric(resultado_paso1['latitu
resultado_paso1['longitud'] = pd.to_numeric(resultado_paso1['longi

# 4. Verificamos plausibilidad territorial (Lat: 13 a 15, Lon: -91
condicion_valida = (
    (resultado_paso1['latitud'] >= 13) & (resultado_paso1['latitud
    (resultado_paso1['longitud'] >= -91) & (resultado_paso1['longi
)

geo_validado = resultado_paso1[condicion_valida].copy()
registros_excluidos = resultado_paso1[~condicion_valida].copy()
```

```
=== SEDES VALIDADAS ===
```

...	sede_id	nombre_sede	municipio	region	latitud	longitud	d
0	S01	Solar Ahuachapán	Ahuachapán	Occidente	13.9214	-89.8453	
1	S02	Solar Santa Ana	Santa Ana	Occidente	13.9942	-89.5597	
2	S03	Solar Sonsonate	Sonsonate	Occidente	13.7189	-89.7241	
3	S04	Solar San Salvador	San Salvador	Centro	13.6929	-89.2182	
4	S05	Solar Santa Tecla	Santa Tecla	Centro	13.6769	-89.2797	
5	S06	Solar Cojutepeque	Cojutepeque	Centro	13.7179	-88.9331	
6	S07	Solar San Miguel	San Miguel	Oriente	13.4837	-88.1779	
7	S08	Solar Usulután	Usulután	Oriente	13.3500	-88.4461	

```
:== SEDES EXCLUIDAS (Error de ubicación) ===
```

	sede_id	nombre_sede	municipio	region	latitud	longitud	defici
8	S09	Solar La Unión	La Unión	Oriente	13.3369	-67.844	

- Exportar geo_validado.csv con las columnas sede_id,latitud,longitud,deficit_total_kwh. Crear la capa de puntos en ArcGIS Map Viewer.

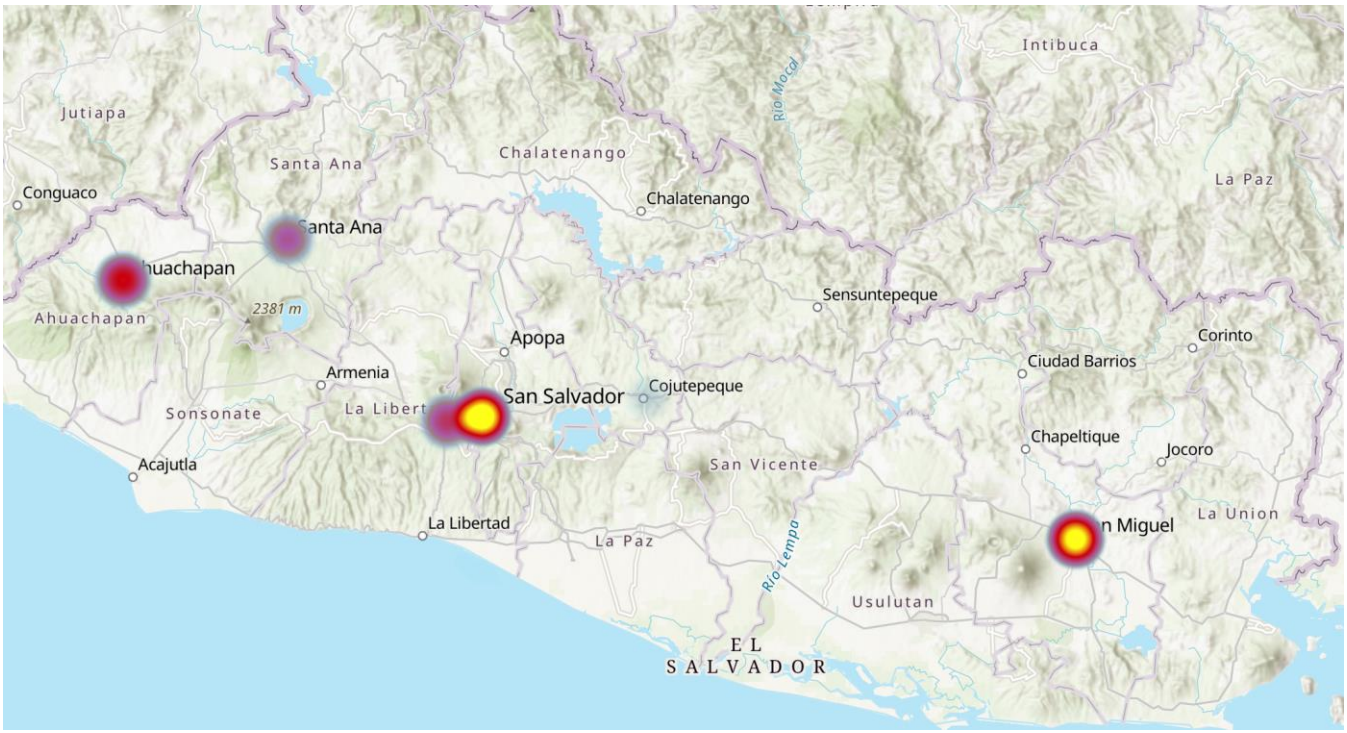
```
# Exportamos
cols_exportar = ['sede_id', 'latitud', 'longitud', 'deficit_total_kwh']
geo_validado_limpio = geo_validado[cols_exportar]
geo_validado_limpio.to_csv('geo_validado.csv', index=False, float_format='%.2f')

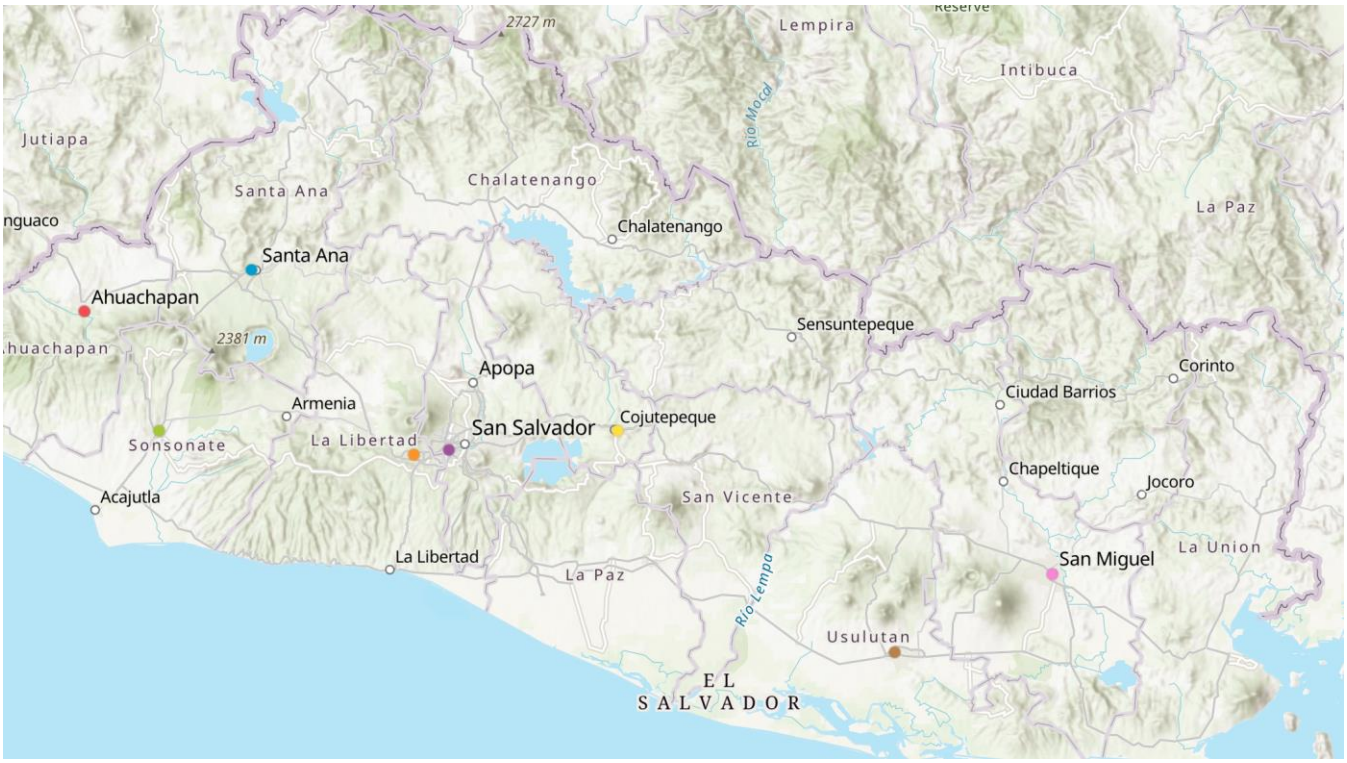
print("\n--- DATOS FINALES A EXPORTAR ---")
print(geo_validado_limpio)
```

```
--- TIPO DE DATOS ---
objeto sede_id
latitud float64
longitud float64
déficit_total_kWh float64
dtype: objeto

--- DATOS FINALES A EXPORTAR ---
  sede_id latitud longitud deficit_total_kwh
0 S01 13.9214 -89.8453 36624.22
1 S02 13,9942 -89,5597 17892,47
2 S03 13.7189 -89.7241 -24670.25
3 S04 13.6929 -89.2182 77336.41
4 S05 13.6769 -89.2797 24360.73
5 S06 13.7179 -88.9331 3494.84
6 S07 13.4837 -88.1779 69364.49
7 S08 13.3500 -88.4461 -13400.66
```

- Mostrar dos vistas del mismo análisis: una simbología por magnitud del déficit y otra exploración filtrada por región o estilizada por categoría; configurar pop-up informativo y capturar mapa_vista1.png y mapa_vista2.png.





- Registrar dos hallazgos territoriales concretos, distinguiendo observación de explicación causal.
  Hallazgo 1: Concentración crítica de déficit energético en zonas metropolitanas y centrales
  Observación (`mapa_vista1.png`): Las sedes ubicadas en la zona central y oriental del país (específicamente **S04** con 77,336.41 kWh y **S07** con 69,364.49 kWh) concentran los valores de déficit energético más elevados de toda la red, superando ampliamente al resto de las estaciones.
  Explicación Causal: Este comportamiento responde a una alta densidad operativa, administrativa e institucional en los puntos céntricos, donde el consumo diario de las instalaciones desborda por completo la capacidad de generación local, requiriendo un abastecimiento externo masivo desde la red nacional.
  Hallazgo 2: Existencia de excedentes energéticos (superávit) en sedes específicas
  Observación (`mapa_vista2.png`): Las sedes **S03** (-24,670.25 kWh) y **S08** (-13,400.66 kWh) presentan valores negativos en el balance de déficit, lo que evidencia que su generación operativa es superior a su demanda real.
  Explicación Causal: Esta condición se origina debido a que dichas sedes poseen fuentes de autogeneración o sistemas con menor presión de consumo diario, generando un superávit de energía que podría aprovecharse estratégicamente para inyectar a zonas vecinas con escasez o déficit crítico.

Justificación técnica (máximo 5 líneas): indique cuántas sedes recibieron validación, cuántas se excluyeron y qué sede presenta el mayor déficit válido. Explique el riesgo de mostrar coordenadas erróneas.

Se validaron 8 sedes geográficamente y se excluyó 1 sede (S09) por un error de longitud fuera de rango. La sede con mayor déficit válido es S04 (Solar San Salvador) con 77,336.41 kWh

Mostrar coordenadas erróneas genera distorsiones espaciales graves, ubicando erróneamente los activos fuera de la región operativa real y comprometiendo la toma de decisiones gerenciales

Parte 3. Reportería gerencial automatizada — 20 puntos

Problema: Cada semana se solicita un informe actualizado. No se permite copiar y pegar manualmente cifras que podrían variar.

```
%%writefile generar_reporte.py
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from pathlib import Path
from datetime import datetime
import json
import hashlib
from jinja2 import Template
import base64
from docx import Document
from docx.shared import Inches
from docx.enum.text import WD_ALIGN_PARAGRAPH
from weasyprint import HTML
import warnings

warnings.filterwarnings('ignore')

def ejecutar_pipeline():
    BASE = Path.cwd() / 'EP2_Equipo4'

    carpetas = [
        'datos',
        'datos_procesados',
        'reportes',
        'ai_modelo',
        'evidencias',
    ]
    for c in carpetas:
        (BASE / c).mkdir(parents=True, exist_ok=True)

    ruta_csv_origen = BASE / 'datos' / 'operacion_sedes.csv'

    if not ruta_csv_origen.exists():
        print(f"Error: No se encontró el archivo en {ruta_csv_origen}")
        return

    sha256_fuente = hashlib.sha256(ruta_csv_origen.read_bytes()).hexdigest()

    df = pd.read_csv(ruta_csv_origen)
    df['fecha'] = pd.to_datetime(df['fecha'])
    df['deficit_kwh'] = np.maximum(0, df['demanda_kwh'] - df['generacion_kwh'])

    registros_entrada = len(df)

    total_gen = float(df['generacion_kwh'].sum())
    total_dem = float(df['demanda_kwh'].sum())
    total_def = float(df['deficit_kwh'].sum())
    cobertura_val = float(100 * (1 - total_def / total_dem) if total_dem > 0 else 0)

    periodo_str = f"{df['fecha'].min().strftime('%d/%m/%Y')} -
{df['fecha'].max().strftime('%d/%m/%Y')}"
```

```
    df_region = df.groupby('region').agg(
        generacion=('generacion_kwh', 'sum'),
        demanda=('demanda_kwh', 'sum'),
        deficit=('deficit_kwh', 'sum')
    ).reset_index()
    df_region['cobertura'] = 100 * (1 - df_region['deficit'] / df_region['demanda'])
    df_region = df_region.sort_values('deficit', ascending=False)
    region_top = df_region.iloc[0]['region']

    df_kpis_csv = pd.DataFrame([
        {'indicador': 'energia_total_kwh', 'valor': round(total_gen, 2)},
        {'indicador': 'demanda_total_kwh', 'valor': round(total_dem, 2)},
        {'indicador': 'deficit_total_kwh', 'valor': round(total_def, 2)}
    ])
    ruta_reporte_kpis = BASE / 'datos_procesados' / 'reporte_kpis.csv'
    df_kpis_csv.to_csv(ruta_reporte_kpis, index=False)

    metadata = {
        'fuente': 'operacion_sedes.csv',
        'registros_entrada': registros_entrada,
        'sha256_fuente': sha256_fuente,
        'fecha_generacion': datetime.now().isoformat(),
        'periodo': periodo_str,
        'kpis': {
            'energia_total_kwh': round(total_gen, 2),
            'demanda_total_kwh': round(total_dem, 2),
            'deficit_total_kwh': round(total_def, 2),
            'cobertura_pct': round(cobertura_val, 2)
        }
    }
    ruta_metadatos = BASE / 'datos_procesados' / 'metadatos.json'
    ruta_metadatos.write_text(json.dumps(metadata, indent=2, ensure_ascii=False),
encoding='utf-8')

    plt.rcParams['figure.figsize'] = (12, 5)
    plt.rcParams['font.size'] = 11
    plt.rcParams['axes.spines.top'] = False
    plt.rcParams['axes.spines.right'] = False

    fig, ax = plt.subplots(figsize=(12, 5))
    df_temporal = df.groupby('fecha')[['generacion_kwh',
'demanda_kwh']].sum().reset_index()
    ax.plot(df_temporal['fecha'], df_temporal['generacion_kwh'], label='Generación
(kWh)', color='#2ca02c', linewidth=2.5)
    ax.plot(df_temporal['fecha'], df_temporal['demanda_kwh'], label='Demanda (kWh)',
color='#1f77b4', linewidth=2.5)
```

```
    ax.set_title('Evolución Temporal: Generación vs Demanda', fontsize=14,
fontweight='bold', pad=15)
    ax.set_xlabel('Fecha', fontsize=11)
    ax.set_ylabel('kWh', fontsize=11)
    ax.legend()
    ax.grid(True, alpha=0.3, linestyle='--')
    plt.tight_layout()

    ruta_grafico = BASE / 'reportes' / 'grafico.png'
    plt.savefig(ruta_grafico, dpi=150, bbox_inches='tight', facecolor='white')
    plt.close()

    plantilla_contenido = """
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Informe Gerencial - GreenEnergy Analytics</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; color: #333; line-height:
1.6; }
        h1 { color: #1a5490; border-bottom: 3px solid #1a5490; padding-bottom: 10px; }
        h2 { color: #2c7bb6; margin-top: 30px; border-bottom: 1px solid #ddd; padding-
bottom: 5px; }
        h3 { color: #444; margin-top: 20px; }
        .portada { text-align: center; margin-bottom: 40px; padding: 25px; background:
#f0f7ff; border-radius: 8px; }
        .kpi-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 15px;
margin: 20px 0; }
        .kpi-card { background: #e8f5e9; padding: 15px; border-left: 4px solid
#2e7d32; border-radius: 4px; }
        .kpi-valor { font-size: 20px; font-weight: bold; color: #1b5e20; }
        .kpi-label { font-size: 11px; color: #555; text-transform: uppercase; margin-
bottom: 5px; }
        .hallazgo { background: #f9f9f9; padding: 15px; border-left: 4px solid
#ff9800; margin: 15px 0; border-radius: 4px; }
        .limitacion { background: #fff3cd; padding: 10px; border-left: 4px solid
#ffc107; margin: 10px 0; border-radius: 4px; }
        img { max-width: 100%; margin: 10px 0; border: 1px solid #eee; border-radius:
4px; }
        .pie { font-size: 11px; color: #999; margin-top: 40px; border-top: 1px solid
#ddd; padding-top: 10px; text-align: center; }
    </style>
</head>
<body>
<div class="portada">
    <h1>INFORME GERENCIAL - GREENENERGY ANALYTICS</h1>
```

```
    <h2>Gestión Energética de Sedes (El Salvador)</h2>
    <p><strong>Periodo:</strong> {{ periodo }}</p>
    <p><strong>Fecha de generación:</strong> {{ fecha_generacion }}</p>
</div>
<h2>1. Resumen Ejecutivo</h2>
<p>El presente informe analiza el desempeño energético de GreenEnergy Analytics en sus
9 sedes durante el periodo {{ periodo }}.</p>
<h3>Indicadores Principales</h3>
<div class="kpi-grid">
    <div class="kpi-card"><div class="kpi-label">Energía Generada Total</div><div
class="kpi-valor">{{ "{:,.2f}".format(kpis.energia_total_kwh) }} kWh</div></div>
    <div class="kpi-card"><div class="kpi-label">Demanda Total</div><div class="kpi-
valor">{{ "{:,.2f}".format(kpis.demanda_total_kwh) }} kWh</div></div>
    <div class="kpi-card"><div class="kpi-label">Déficit Energético Total</div><div
class="kpi-valor">{{ "{:,.2f}".format(kpis.deficit_total_kwh) }} kWh</div></div>
</div>
<h2>2. Análisis Temporal</h2>
<h3>2.1 Evolución de Generación vs Demanda</h3>
<img src="{{ ruta_grafico }}" alt="Gráfico Operativo">
<h2>3. Hallazgos Principales</h2>
<div class="hallazgo">
    <h3>1. Concentración de Déficit Operativo</h3>
    <p><strong>Evidencia:</strong> Déficit total acumulado de {{
"{:,.2f}".format(kpis.deficit_total_kwh) }} kWh (Cobertura global: {{
"%.2f"|format(kpis.cobertura_pct) }}%).</p>
    <p><strong>Interpretación:</strong> La región {{ region_top }} concentra los
mayores requerimientos de energía externa según el ordenamiento por déficit.</p>
</div>
<h2>4. Recomendación</h2>
<p><strong>Acción sugerida:</strong> Priorizar refuerzos de capacidad limpia en las
sedes con mayor déficit identificado.</p>
<h2>5. Limitación del Análisis</h2>
<div class="limitacion"><p><strong>[!] Limitación:</strong> Los datos corresponden a
180 días de operación sintética y no incluyen imprevistos meteorológicos
severos.</p></div>
<div class="pie"><p>Pipeline de reportería automatizada - GreenEnergy
Analytics</p></div>
</body>
</html>
"""

    ruta_plantilla = BASE / 'reportes' / 'plantilla.html'
    ruta_plantilla.write_text(plantilla_contenido, encoding='utf-8')

    def imagen_a_base64(ruta_img):
        with open(ruta_img, 'rb') as f:
            return f"{base64.b64encode(f.read()).decode()}"
```

```
    kpis_obj = type('KPIs', (), {
        'energia_total_kwh': total_gen,
        'demanda_total_kwh': total_dem,
        'deficit_total_kwh': total_def,
        'cobertura_pct': cobertura_val
    })()

    datos_template = {
        'periodo': periodo_str,
        'fecha_generacion': datetime.now().strftime('%d/%m/%Y %H:%M'),
        'kpis': kpis_obj,
        'ruta_grafico': imagen_a_base64(ruta_grafico),
        'region_top': region_top,
    }

    template_cargada = Template(ruta_plantilla.read_text(encoding='utf-8'))
    html_renderizado = template_cargada.render(**datos_template)

    ruta_html = BASE / 'reportes' / 'informe.html'
    ruta_html.write_text(html_renderizado, encoding='utf-8')

    ruta_pdf = BASE / 'reportes' / 'informe.pdf'
    HTML(string=html_renderizado).write_pdf(str(ruta_pdf))

    doc = Document()
    titulo = doc.add_heading('INFORME GERENCIAL ENERGÉTICO', 0)
    titulo.alignment = WD_ALIGN_PARAGRAPH.CENTER
    subtitulo = doc.add_heading('GreenEnergy Analytics', level=1)
    subtitulo.alignment = WD_ALIGN_PARAGRAPH.CENTER

    doc.add_paragraph(f'Periodo: {periodo_str}')
    doc.add_paragraph(f'Fecha de generación: {datetime.now().strftime("%d/%m/%Y
%H:%M")}')

    doc.add_heading('1. Resumen Ejecutivo e Indicadores', level=2)
    tabla_kpis = doc.add_table(rows=4, cols=2)
    tabla_kpis.style = 'Light Grid Accent 1'
    datos_docx_kpis = [
        ('Energía Generada Total', f'{total_gen:,.2f} kWh'),
        ('Demanda Total', f'{total_dem:,.2f} kWh'),
        ('Déficit Total', f'{total_def:,.2f} kWh'),
        ('Cobertura Global %', f'{cobertura_val:.2f}%')
    ]
    for i, (label, valor) in enumerate(datos_docx_kpis):
        tabla_kpis.rows[i].cells[0].text = label
        tabla_kpis.rows[i].cells[1].text = valor
```

```
    doc.add_heading('2. Gráfico Operativo', level=2)
    doc.add_picture(str(ruta_grafico), width=Inches(6))

    doc.add_heading('3. Hallazgos y Recomendaciones', level=2)
    doc.add_paragraph(f'La región {region_top} concentra el mayor déficit operativo.')
    doc.add_paragraph('Recomendación: Priorizar capacidad limpia en sedes críticas.')

    ruta_docx = BASE / 'reportes' / 'informe.docx'
    doc.save(str(ruta_docx))

    print("¡Pipeline ejecutado con éxito!")

if __name__ == '__main__':
    ejecutar_pipeline()
```

- Crear generar_reporte.py que cargue el archivo de operaciones y calcule con Pandas energía generada, demanda y déficit totales.

```
# Cargar el dataset con Pandas y calcular el déficit por registro
df = pd.read_csv(ruta_csv_origen)
df['fecha'] = pd.to_datetime(df['fecha'])
df['deficit_kwh'] = np.maximum(0, df['demanda_kwh'] - df['generacion_kwh'])

# Calcular los totales oficiales requeridos
total_gen = float(df['generacion_kwh'].sum())
total_dem = float(df['demanda_kwh'].sum())
total_def = float(df['deficit_kwh'].sum())
cobertura_val = float(100 * (1 - total_def / total_dem) if total_dem > 0 else 0)
```

- Generar un gráfico en PNG con datos del caso, una plantilla Jinja2 llamada plantilla.html y su salida informe.html.

```
# Generar y guardar el gráfico en PNG:
fig, ax = plt.subplots(figsize=(12, 5))
df_temporal = df.groupby('fecha')[['generacion_kwh',
'demanda_kwh']].sum().reset_index()
ax.plot(df_temporal['fecha'], df_temporal['generacion_kwh'], label='Generación (kWh)',
color='#2ca02c', linewidth=2.5)
ax.plot(df_temporal['fecha'], df_temporal['demanda_kwh'], label='Demanda (kWh)',
color='#1f77b4', linewidth=2.5)
ax.set_title('Evolución Temporal: Generación vs Demanda', fontsize=14,
fontweight='bold', pad=15)
ax.set_xlabel('Fecha', fontsize=11)
ax.set_ylabel('kWh', fontsize=11)
ax.legend()
ax.grid(True, alpha=0.3, linestyle='--')
plt.tight_layout()

ruta_grafico = BASE / 'reportes' / 'grafico.png'
plt.savefig(ruta_grafico, dpi=150, bbox_inches='tight', facecolor='white')
plt.close()
```

```
#Crear plantilla.html y generar informe.html:
# Definir el contenido de la plantilla Jinja2
plantilla_contenido = """<!DOCTYPE html><html>...</html>"""
ruta_plantilla = BASE / 'reportes' / 'plantilla.html'
ruta_plantilla.write_text(plantilla_contenido, encoding='utf-8')

# Renderizar la salida informe.html
template_cargada = Template(ruta_plantilla.read_text(encoding='utf-8'))
html_renderizado = template_cargada.render(**datos_template)

ruta_html = BASE / 'reportes' / 'informe.html'
ruta_html.write_text(html_renderizado, encoding='utf-8')
```

- Exportar el informe a informe.docx o informe.pdf usando las técnicas trabajadas en la Guía 7. Incorporar título, periodo, 3 KPIs, 1 gráfico, 1 hallazgo, 1 recomendación y 1 limitación real.

```
# Exportar a PDF con WeasyPrint:
from weasyprint import HTML
ruta_pdf = BASE / 'reportes' / 'informe.pdf'
HTML(string=html_renderizado).write_pdf(str(ruta_pdf))
```

```
# Exportar a DOCX con Python-Docx (con tabla de KPIs y elementos estructurados):
from docx import Document
from docx.shared import Inches
from docx.enum.text import WD_ALIGN_PARAGRAPH

doc = Document()
titulo = doc.add_heading('INFORME GERENCIAL ENERGÉTICO', 0)
titulo.alignment = WD_ALIGN_PARAGRAPH.CENTER

# Adición de periodo, tabla con los 3 KPIs, imagen del gráfico, hallazgos,
recomendaciones y limitaciones
doc.add_paragraph(f'Periodo: {periodo_str}')
#Adición de tabla y secciones en docx
ruta_docx = BASE / 'reportes' / 'informe.docx'
doc.save(str(ruta_docx))
```

- Generar reporte_kpis.csv con indicador,valor para los tres KPIs oficiales y metadatos.json que declare fuente, registros_entrada y sha256_fuente del archivo operacion_sedes.csv.

```
# Para reporte_kpis.csv:

df_kpis_csv = pd.DataFrame([
    {'indicador': 'energia_total_kwh', 'valor': round(total_gen, 2)},
```

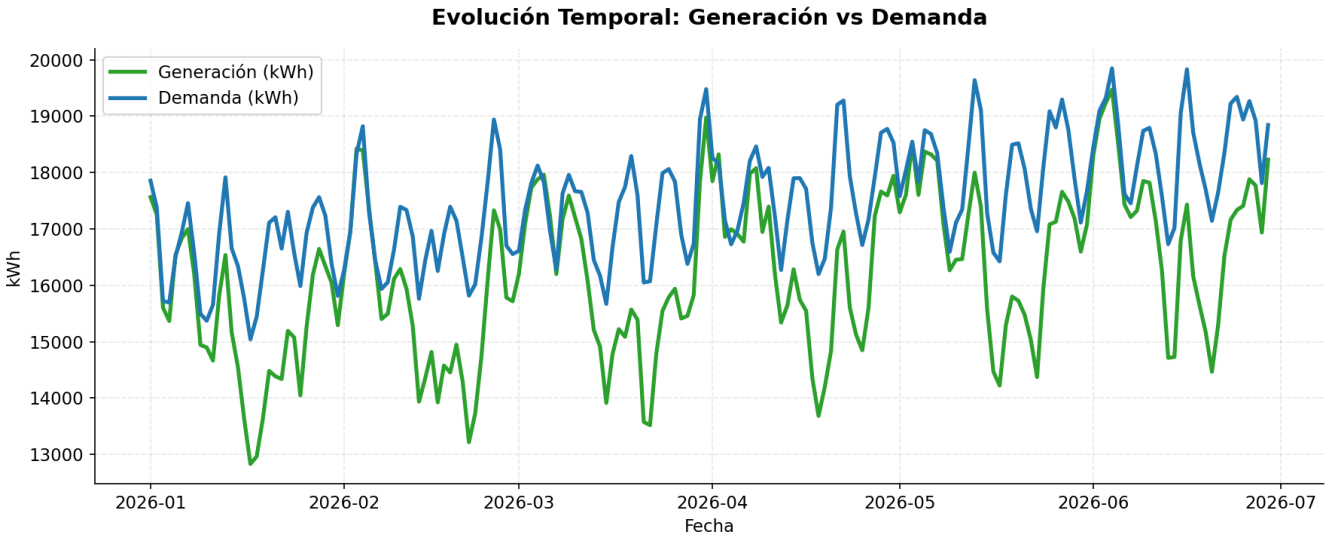
```
    {'indicador': 'demanda_total_kwh', 'valor': round(total_dem, 2)},
    {'indicador': 'deficit_total_kwh', 'valor': round(total_def, 2)}
])
ruta_reporte_kpis = BASE / 'datos_procesados' / 'reporte_kpis.csv'
df_kpis_csv.to_csv(ruta_reporte_kpis, index=False)
```

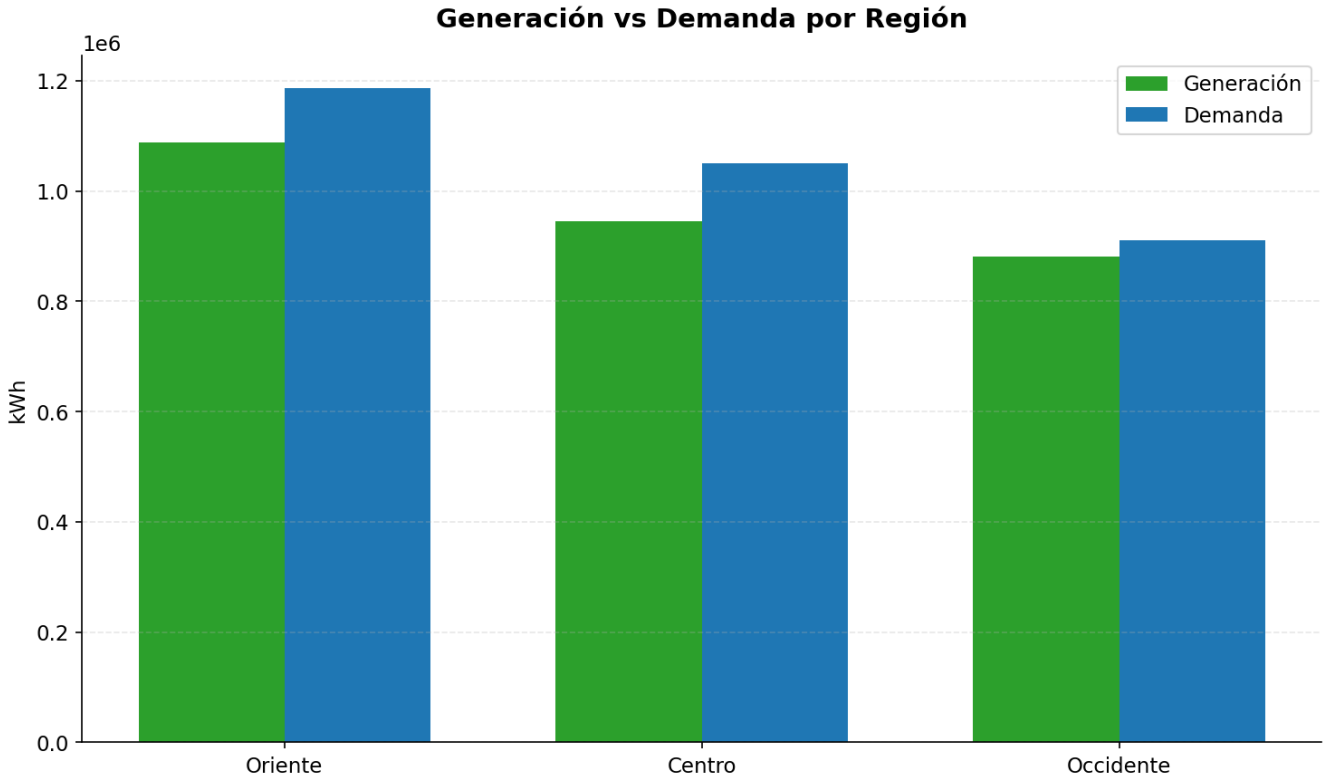
```
# Para metadatos.json (calculando el hash SHA-256 y conteo de filas):
sha256_fuente = hashlib.sha256(ruta_csv_origen.read_bytes()).hexdigest()
registros_entrada = len(df)

metadata = {
    'fuente': 'operacion_sedes.csv',
    'registros_entrada': registros_entrada,
    'sha256_fuente': sha256_fuente,
    'fecha_generacion': datetime.now().isoformat(),
    'periodo': periodo_str
}
ruta_metadatos = BASE / 'datos_procesados' / 'metadatos.json'
ruta_metadatos.write_text(json.dumps(metadata, indent=2, ensure_ascii=False),
encoding='utf-8')
```

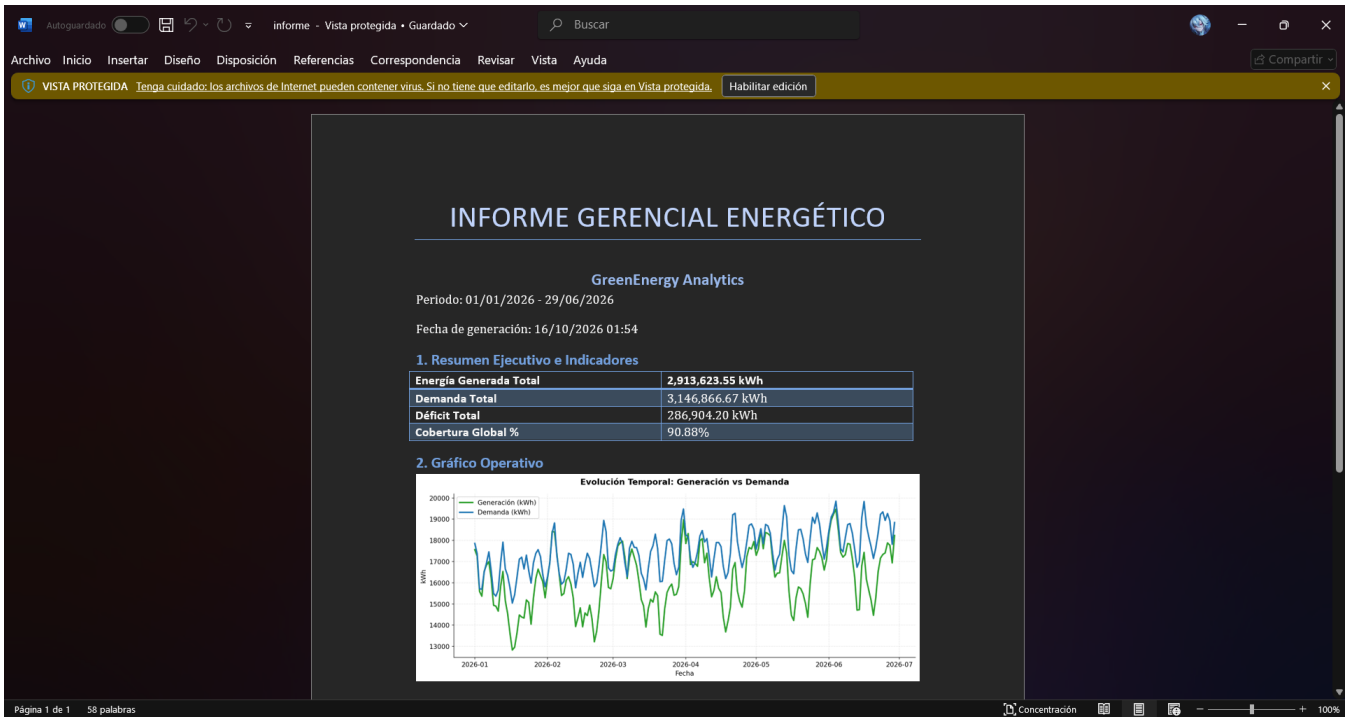
- Demostrar que la salida puede regenerarse ejecutando el mismo script, sin editar manualmente el informe.

```
# Celda final vacía para control y ejecución del pipeline automatizado
!python generar_reporte.py
print("¡Se ejecuto con éxito!")
```

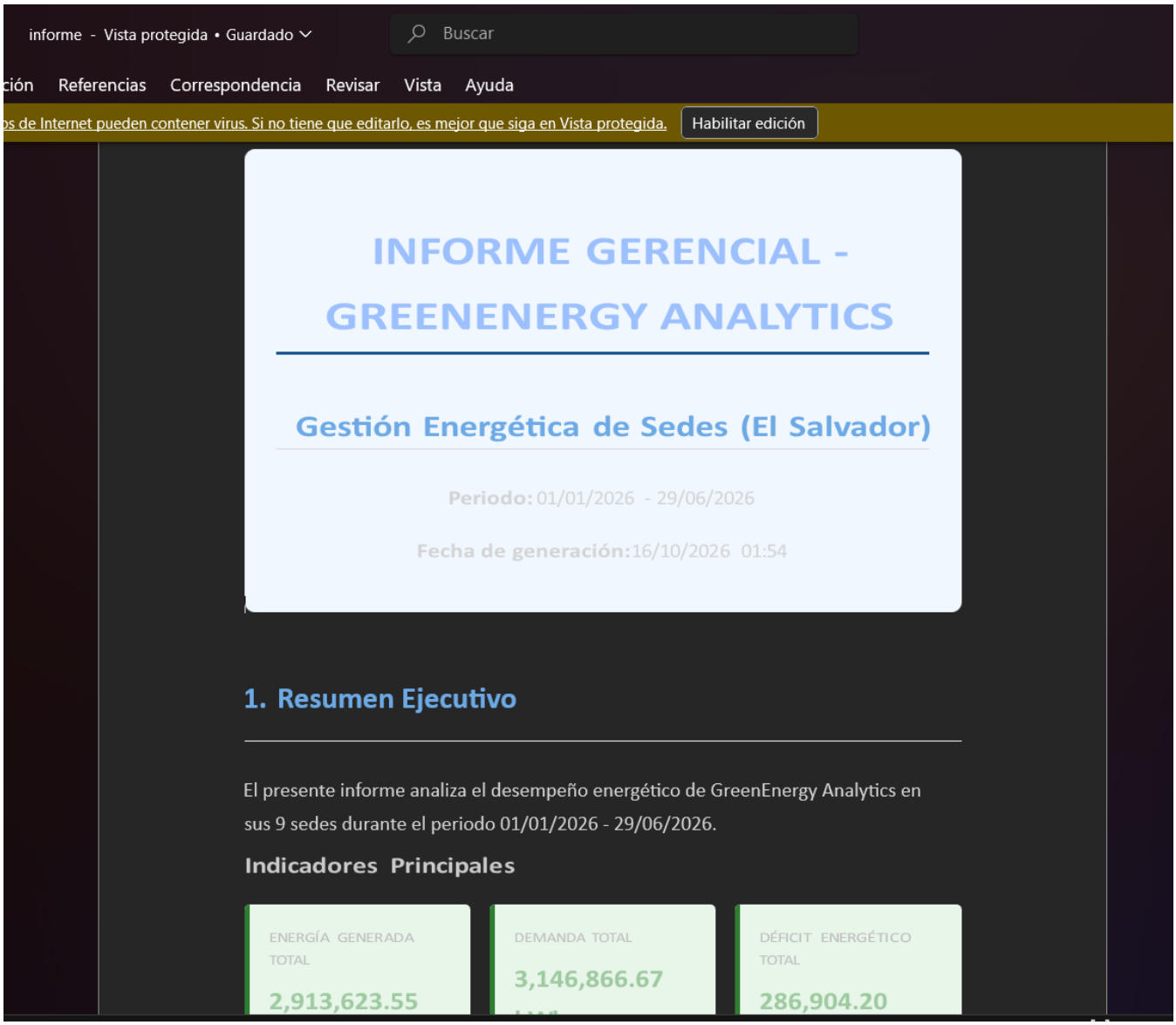

Evolución Temporal: Generación vs Demanda
Generación (kWh)
Demanda (kWh)
kWh
20000
19000
18000
17000
16000
15000
14000
13000
2026-01
2026-02
2026-03
2026-04
2026-05
2026-06
2026-07
Fecha


Generación vs Demanda por Región
1e6
1.2
1.0
0.8
0.6
0.4
0.2
0.0
kWh
Generación
Demanda
Oriente
Centro
Occidente

Docx


Autoguardado
informe - Vista protegida • Guardado
Buscar
Archivo Inicio Insertar Diseño Disposición Referencias Correspondencia Revisar Vista Ayuda
Compartir
VISTA PROTEGIDA Tenga cuidado: los archivos de Internet pueden contener virus. Si no tiene que editarlo, es mejor que siga en Vista protegida.
Habilitar edición
INFORME GERENCIAL ENERGÉTICO
GreenEnergy Analytics
Periodo: 01/01/2026 - 29/06/2026
Fecha de generación: 16/10/2026 01:54
1. Resumen Ejecutivo e Indicadores
Energía Generada Total 2,913,623.55 kWh
Demanda Total 3,146,866.67 kWh
Déficit Total 286,904.20 kWh
Cobertura Global % 90.88%
2. Gráfico Operativo
Evolución Temporal: Generación vs Demanda
Generación (kWh)
Demanda (kWh)
kWh
Fecha
Página 1 de 1
58 palabras
Concentración
100%

PDF


informe - Vista protegida • Guardado
Buscar
ción Referencias Correspondencia Revisar Vista Ayuda
os de Internet pueden contener virus. Si no tiene que editarlo, es mejor que siga en Vista protegida.
Habilitar edición
INFORME GERENCIAL - GREENENERGY ANALYTICS
Gestión Energética de Sedes (El Salvador)
Periodo: 01/01/2026 - 29/06/2026
Fecha de generación:16/10/2026 01:54
1. Resumen Ejecutivo
El presente informe analiza el desempeño energético de GreenEnergy Analytics en sus 9 sedes durante el periodo 01/01/2026 - 29/06/2026.
Indicadores Principales
ENERGÍA GENERADA TOTAL
2,913,623.55
DEMANDA TOTAL
3,146,866.67
DÉFICIT ENERGÉTICO TOTAL
286,904.20

generar_informe.yml

C: > Users > david > Downloads > generar_informe.yml

```yaml

name: Generar Informe Automático

on:
  schedule:
    - cron: '0 8 * * 1'  # Todos los lunes a las 8 AM
  workflow_dispatch:

jobs:
  generar-informe:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'
      - name: Instalar dependencias
        run: pip install pandas matplotlib jinja2 weasyprint python-docx
      - name: Generar informe
        run: python scripts/generar_informe.py
      - name: Subir artefactos
        uses: actions/upload-artifact@v3
        with:
          name: informe-generado
          path: exportaciones/

```

Justificación técnica (máximo 5 líneas): describa qué cambia al regenerar el informe, qué dato comprueba su concordancia con el dashboard y cuál es la principal limitación operativa del caso.

Justificacion: *Al volver a crear el informe, se actualizan automáticamente las fechas, los gráficos y las tablas sin que se tenga que mover nada. Los datos se comprueban comparándolos con un archivo de control y una firma digital que verifica que la información original no ha sido alterada. El único detalle es que, como todo se basa en una simulación de medio año, el sistema no está preparado para imprevistos por clima extremo o desastres repentinos.*

Parte 4. Fundamentos de predicción con IA — 30 puntos

Problema: Se requiere estimar la demanda energética del siguiente día con un modelo inicial, sin anticipar información futura ni presentar resultados como certezas.

- Abrir demanda_diaria.csv, ordenar cronológicamente, revisar nulos y definir y = demanda_kwh.

```
# # Cargar el archivo de demanda diaria
df = pd.read_csv('demanda_diaria.csv')

# Convertir fecha a tipo fecha y ordenar cronológicamente
df['fecha'] = pd.to_datetime(df['fecha'])
df = df.sort_values('fecha').reset_index(drop=True)
```

```
# Revisar valores nulos por columna
print(df.isna().sum())

# Variable objetivo demanda del día
y = df['demanda_kwh']
```

```
fecha              0
demanda_kwh        0
evento_programado  0
dtype: int64
```

- Crear X con dia_semana, mes, evento_programado, lag_1, lag_7 y media_movil_7. Las tres últimas características se derivan exclusivamente de demanda previa; media_movil_7 debe excluir la demanda del mismo día mediante shift(1).

```
# Crear variables temporales
df['dia_semana'] = df['fecha'].dt.dayofweek
df['mes'] = df['fecha'].dt.month
# Creación de variables de rezago y media móvil sobre la variable real de demanda
df['lag_1'] = df['demanda_kwh'].shift(1)
df['lag_7'] = df['demanda_kwh'].shift(7)
df['media_movil_7'] = df['demanda_kwh'].shift(1).rolling(window=7).mean()

# Limpiar los valores nulos iniciales generados por los rezagos
df = df.dropna().reset_index(drop=True)

print("¡Dataset real cargado y preparado exitosamente!")
display(df.head())
```

```
¡Dataset real cargado y preparado exitosamente!
```

	fecha	demanda_kwh	evento_programado	dia_semana	mes	lag_1	lag_7	media_movil_7
0	2026-01-08	16568.10	0	3	1	17456.69	17856.31	16792.854286
1	2026-01-09	15495.83	0	4	1	16568.10	17379.10	16608.824286
2	2026-01-10	15370.24	0	5	1	15495.83	15720.70	16339.785714
3	2026-01-11	15652.73	0	6	1	15370.24	15694.05	16289.720000
4	2026-01-12	16919.53	1	0	1	15652.73	16505.77	16283.817143

```
#Definir las 6 características exactas exigidas por el enunciado
features = ["lag_1", "lag_7", "media_movil_7", "dia_semana", "mes",
"evento_programado"]
X = df[features]
y = df["demanda_kwh"]
```

- Eliminar las filas iniciales sin historial, realizar división cronológica 80/20, sin mezclar aleatoriamente.

```
# Calcular el punto de corte para la partición cronológica
train_size = int(len(df) * 0.8)

# Dividir los datos de entrenamiento y prueba y no lo mezclamos
X_train = X.iloc[:train_size]
X_test = X.iloc[train_size:]
y_train = y.iloc[:train_size]
y_test = y.iloc[train_size:]
```

```
# Guardamos las fechas correspondientes al conjunto de prueba
fechas_test = df['fecha'].iloc[train_size:]

print(f"Total de registros procesados: {len(df)}")
print(f"Tamaño de X_train: {len(X_train)}")
print(f"Tamaño de X_test (y sus fechas): {len(X_test)}")
```

```
Total de registros procesados: 173
Tamaño de X_train: 138
Tamaño de X_test (y sus fechas): 35
```

- Entrenar DummyRegressor(strategy="mean") como baseline y LinearRegression() como modelo principal (Scikit-Learn).

```
# Entrenar el Baseline
dummy_model = DummyRegressor(strategy="mean")
dummy_model.fit(X_train, y_train)
y_pred_dummy = dummy_model.predict(X_test)
```

```
# Entrenar el Modelo Base
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
```

```
# Calcular métricas del Baseline
mae_dummy = mean_absolute_error(y_test, y_pred_dummy)
rmse_dummy = np.sqrt(mean_squared_error(y_test, y_pred_dummy))
r2_dummy = r2_score(y_test, y_pred_dummy)

# Calcular métricas de la Regresión Lineal
mae_lr = mean_absolute_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_lr = r2_score(y_test, y_pred_lr)
```

```
# Porcentaje de mejora del MAE
mejora_mae = ((mae_dummy - mae_lr) / mae_dummy) * 100

print("=== MÉTRICAS BASELINE (Dummy) ===")
print(f"MAE: {mae_dummy:.4f} | RMSE: {rmse_dummy:.4f} | R2: {r2_dummy:.4f}\n")

print("=== MÉTRICAS MODELO BASE (Linear Regression) ===")
print(f"MAE: {mae_lr:.4f} | RMSE: {rmse_lr:.4f} | R2: {r2_lr:.4f}\n")

print(f"Porcentaje de mejora del MAE respecto al baseline: {mejora_mae:.2f}%")
```

```
=== MÉTRICAS BASELINE (Dummy) ===
MAE: 1194.5730 | RMSE: 1391.6764 | R2: -1.9384

=== MÉTRICAS MODELO BASE (Linear Regression) ===
MAE: 298.7650 | RMSE: 378.7246 | R2: 0.7824

Porcentaje de mejora del MAE respecto al baseline: 74.99%
```

- Exportar predicciones.csv con columnas fecha,real,baseline,prediccion, en orden temporal para el conjunto de prueba, y metricas.json con MAE del baseline, MAE y RMSE del modelo, R^2 y porcentaje de mejora del MAE.

```
import json

# Creamos el diccionario plano con los nombres de claves exactos del contrato
metricas_dict = {
    "mae_baseline": float(mae_dummy),
    "mae_modelo": float(mae_lr),
    "rmse_modelo": float(rmse_lr),
    "r2_modelo": float(r2_lr),
    "mejora_mae_pct": float(mejora_mae)
}

with open("metricas.json", "w") as f:
    json.dump(metricas_dict, f, indent=4)
```

```
# generamos el DataFrame con los nombres de columnas exactos y formato de fecha limpio
df_predicciones = pd.DataFrame({
    "fecha": pd.to_datetime(fechas_test).dt.strftime('%Y-%m-%d'),
    "real": y_test.values,
    "baseline": y_pred_dummy,
    "prediccion": y_pred_lr
})

df_predicciones.to_csv("predicciones.csv", index=False)

print("¡Archivos 'metricas.json' y 'predicciones.csv' actualizados con el contrato exacto del EP2!")
```

```
display(df_predicciones.head())
```

```
¡Archivos 'metricas.json' y 'predicciones.csv' actualizados con el contrato exacto del EP2!
```

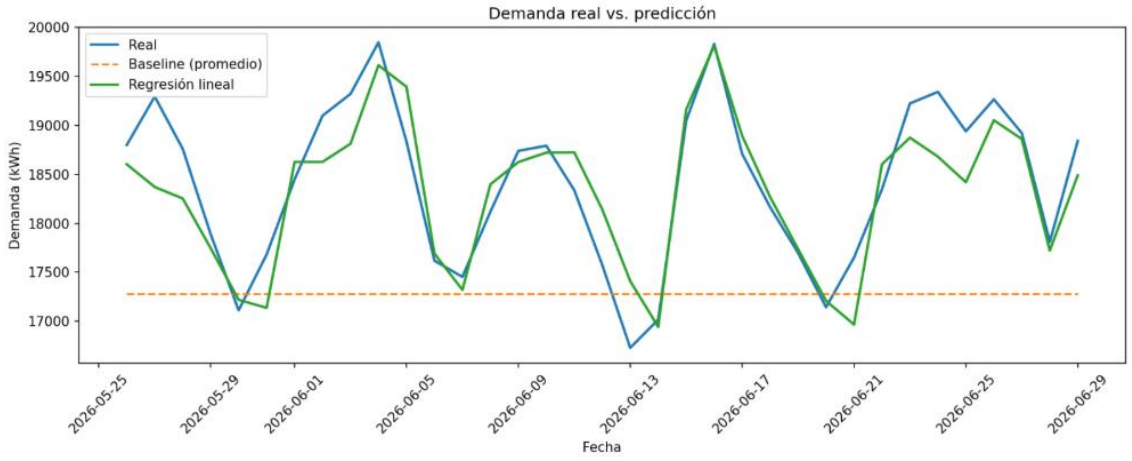
	fecha	real	baseline	prediccion
138	2026-05-26	18800.00	17281.821304	18603.319450
139	2026-05-27	19294.76	17281.821304	18371.300370
140	2026-05-28	18761.74	17281.821304	18254.034120
141	2026-05-29	17890.43	17281.821304	17746.713486
142	2026-05-30	17112.40	17281.821304	17218.995397

- Guardar el notebook ejecutable modelo.ipynb y explicar si el modelo realmente agrega valor frente al baseline.

```
# Imprimir la primera y la última fecha de prueba
print(f"Primera fecha de prueba: {fechas_test.iloc[0]}")
print(f"Última fecha de prueba: {fechas_test.iloc[-1]}")

# Ver tus métricas calculadas
print(f"MAE Baseline: {mae_dummy:.4f}")
print(f"MAE Modelo: {mae_lr:.4f}")
print(f"% de Mejora: {mejora_mae:.2f}%")
```

Imagen de la gráfica



-

Justificacion Tecnica

El conjunto de prueba va del 2026-05-26 al 2026-06-29. El modelo lineal agrega valor real: su MAE es 298.77 frente a 1194.57 del baseline, una mejora del 74.99%. No se entrena con el futuro porque la partición es cronológica y la media móvil usa

shift(1); de lo contrario, la fuga de información sobreestimaría las métricas. Su limitación principal es que la prueba dura solo 35 días y el modelo necesita los registros históricos previos para calcular los rezagos.

7. Contrato de salida para revisión semiautomatizada

Estos nombres, columnas y claves son obligatorios. La revisión automática no ejecutará código estudiantil: comparará los productos contra cálculos independientes realizados sobre los datasets oficiales.

Archivo	Estructura exacta
indicadores.csv	indicador,valor | energia_total_kwh; demanda_total_kwh; deficit_total_kwh; cobertura_pct
geo_validado.csv	sede_id,latitud,longitud,deficit_total_kwh
reporte_kpis.csv	indicador,valor | energia_total_kwh; demanda_total_kwh; deficit_total_kwh
metadatos.json	fuente, registros_entrada, sha256_fuente
predicciones.csv	fecha,real,baseline,prediccion | Fecha AAAA-MM-DD y decimales con punto
metricas.json	mae_baseline, mae_modelo, rmse_modelo, r2_modelo, mejora_mae_pct
equipo.json	{"equipo":"E01","seccion":"IDS221"} (adapte código y sección)
integrantes.csv	carnet,nombre,aporte | 4 o 5 integrantes del equipo

8. Entregables y medio de entrega

Suban UN archivo EP2_EquipoXX.zip al buzón Moodle. El archivo debe contener, como mínimo, los siguientes elementos (se aceptan subcarpetas; no cambien los nombres de los archivos). Este mismo entregable deberá estar en github:

- Identificación: equipo.json, integrantes.csv y justificaciones.md (4 justificaciones identificadas P1, P2, P3 y P4).
- Power BI: dashboard.pbix, vista_bi.png e indicadores.csv.
- ArcGIS: geo_validado.csv, mapa_vista1.png, mapa_vista2.png; enlace de ArcGIS dentro de justificaciones.md cuando el acceso lo permita.
- Reportería: generar_reporte.py, plantilla.html, informe.html, informe.docx o informe.pdf, grafico.png, reporte_kpis.csv y metadatos.json.
- Predicción: modelo.ipynb, predicciones.csv y metricas.json.

REGLA DE EVIDENCIA: Una captura no sustituye el archivo funcional. Un dato correcto copiado en un CSV sin evidencia de proceso no demuestra por sí solo autoría; el docente podrá pedir reproducir un filtro, ejecutar un script o explicar una predicción. El vídeo demostrativo reforzará el entregable

9. Rúbrica de evaluación por competencias

Competencia / parte	Verificación automática	Validación docente	Total
1. Power BI	15: cuatro KPIs reproducidos	10: PBIX, medidas, visuales, filtro y claridad	25
2. ArcGIS	10: sede/posición/déficit válidos	10: dos vistas, estilos, pop-ups, patrones	20
3. Reportería	17: integridad, KPIs, archivos, metadata	3: regeneración y coherencia	20
4. Predicción	25: fechas, baseline, regresión y métricas	5: interpretación y ausencia de fuga	30
Comprobación individual	—	**5: explicación o cambio en vivo por integrante**	5
TOTAL	67 puntos	33 puntos	100

10. Criterios generales y comprobación individual

- Funcionalidad y exactitud: todos los resultados deberán corresponder a los archivos oficiales sin alterar entradas para mejorar artificialmente métricas.
- Reproducibilidad: el informe y el notebook deben ejecutarse; cuando haya fallos, se valorará el intento documentado y la evidencia verificable.
- Integridad y colaboración: cada integrante registra su aporte en integrantes.csv y explica una decisión, fórmula, mapa o métrica propia.
- Comprobación breve: el docente podrá solicitar un cambio de filtro en vivo o una explicación concreta sobre el resultado generado. **El componente individual puede producir notas distintas dentro del equipo.**

- Retroalimentación: el validador identificará criterios superados y pendientes; los puntos visuales, interpretación y autoría los registrará el docente.

Éxito en la evaluación. Prioricen una solución que funciona, se puede comprobar y pueden explicar, antes que un producto visualmente complejo sin evidencia técnica.